대규모 시스템 설계 기초 2 · 제5장

지표 모니터링 및 경보 시스템 설계

1,000만 개 운영 지표를 유실 없이 처리하는 고가용성 파이프라인

인프라 아키텍처 기술 세미나 발표 자료

모니터링은 왜 필요한가?

모니터링의 핵심 목적

인프라 장애를 사전에 감지하고 가용성을 확보

관측성 체계 및 설계 집중 영역

- **관측 가능성(Observability)의 3대 축**
  지표(Metrics) · 로그(Logs) · 분산 추적(Tracing)
- **설계 대상: 수치 기반 운영 지표(Metrics) 집중**
  CPU 사용률, 메모리, 초당 요청 수(RPS) 실시간 감시
- **범위 제외: 로그 및 분산 추적 분리**
  텍스트 검색(ELK)과 서비스 호출 경로는 별도 파이프라인으로 구성



핵심 요구사항 및 구체적 설계 조건

모니터링 인프라 규모

- 대형 IT 기업이 내부에서 사용하는 시스템
- 일간 능동 사용자 수(DAU) 1억 명 지원 대규모 인프라
- 1,000개 서버 풀 × 100대 서버
- 에러 로그, 액세스 로그 수집 기능, 분산 추적은 제외

서버당 100개 지표 수집 / 수집 주기 10초

약 1,000만 개

경보 채널 및 비기능 목표

- **지원 채널:** 이메일 · SMS · 페이저듀티(PagerDuty) · HTTP 웹훅
- **비기능 요구:** 수평적 규모 확장성 · 저지연 질의 · 경보 무누락 보장

데이터 보관 기간 및 단계별 다운샘플링 정책

총 보관 1년

최근 7일 — 수집 원본 해상도 유지

수집한 원본 그대로 보관하여 초 단위 미세 장애 감지 및 즉시 대응

7일 ~ 30일 — 1분 단위 집계 보관

1분 단위 데이터로 롤업(Rollup) 변환 후 30일간 중기 트렌드 분석에 활용

30일 ~ 1년 — 1시간 단위 변환 보관

1시간 단위 데이터로 변환 후 냉동 저장소로 이관하여 장기 보관

시계열(Time-Series) 데이터 및 행 프로토콜

지표 이름

측정 대상을 식별하는 문자열

cpu.load

태그 및 레이블

다차원 질의용 <키:값> 쌍

host:web01, region:us-west

타임스탬프와 값

측정 시각과 지표 수치 쌍

1613707265 : 50

공통 표준 규격

행 프로토콜 (Line Protocol)

시계열 데이터베이스가 대규모 쓰기 연산을 초당 수십만 건 이상 고속 처리하기 위해 채택한 표준 텍스트 포맷입니다.

InfluxDB

Prometheus

Line Protocol 예시 (책 사례 2)

포맷: 지표명 레이블목록 타임스탬프 값

```
CPU.load host=webserver01,region=us-west 1613707265 50
CPU.load host=webserver01,region=us-west 1613707265 62
CPU.load host=webserver02,region=us-west 1613707265 43
CPU.load host=webserver02,region=us-west 1613707265 53
```

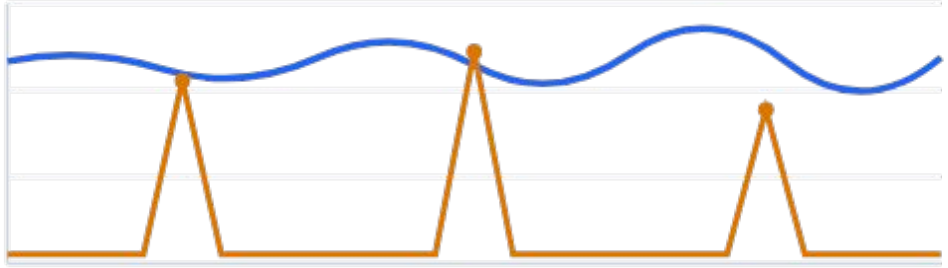
데이터 접근 패턴 분석

데이터 접근 패턴의 극단적 비대칭성

시간대별 트래픽 패턴 비교

쓰기 트래픽 (상시 고부하)

읽기 트래픽 (간헐적 스파이크)



시간 경과 (00:00 → 24:00)

읽기 스파이크: 대시보드 갱신 및 경보 임계치 평가 시점

압도적인 쓰기 트래픽

1,000만 지표가 주기적으로 유입되며, 기존 데이터를 수정하지 않는 순수 추가 전용(Append-only) 연산 지속

일시적인(Spiky) 읽기 트래픽

평소 읽기 부하는 낮으나, 장애 분석이나 경보 규칙 평가 주기에 순간적으로 대량 질의 발생

85%

데이터 접근의 시간 국소성

페이스북 연구: 전체 질의의 85%는 **최근 26시간 이내** 데이터에 집중 → 고속 메모리 캐싱

저장소 엔진 비교 및 최적 솔루션 선정

비교 항목	관계형 DB (MySQL)	범용 NoSQL (Cassandra)	시계열 DB (TSDB)
대규모 쓰기 성능	낮음 (인덱스 병목)	높음 (분산 쓰기)	**극대화 (초당 25만 회+)**
시계열 연산 질의	SQL 복잡도 극심	스키마 튜닝 난이도 높음	**전용 질의어 지원 (Flux/PromQL)**
저장 용량 최적화	수동 파티셔닝 한계	자체 압축 구현 필요	**이중 델타 및 다운샘플링 내장**
최종 평가	적용 불가	운영 부담 큼	**업계 표준 채택 (InfluxDB 등)**

책 2단계 · 그림 5.6 개략적 설계안 구조화

개략적 설계안: 엔드투엔드 파이프라인



설계 핵심: 전용 질의 서비스를 중간에 두어 시계열 DB와 클라이언트(경보·시각화) 사이의 결합도를 낮추고, 추후 저장소 교체 유연성을 확보합니다.

상세 설계 1: 지표 수집 모델: 풀(Pull) vs 푸시(Push)

풀 모델 (Pull Model)

대표: Prometheus

동작 원리 (설명)

중앙의 지표 수집기가 모니터링 대상 서버의 /metrics HTTP 엔드포인트를 주기적으로 직접 호출하여 지표를 긁어오는(Pull) 방식입니다.

애플리케이션 개발 시 지표 수집용 클라이언트 라이브러리 SDK만 포함시킵니다.

핵심 특징 및 요구 기술

• 손쉬운 상태 진단 및 로컬 디버깅

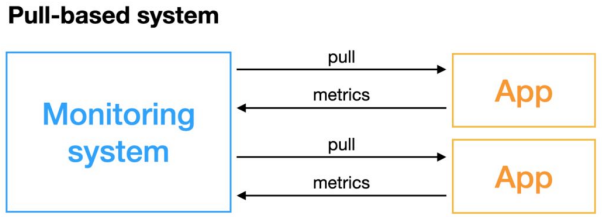
서버 무응답 시 즉각 장애로 판별.
curl http://localhost:8080/metrics 웹 브라우저에서 지표를 바로 확인 가능.

• 서비스 탐색(SDS) 연동 필수

SDS(Service Discovery Service) 중앙 수집기에서 지표를 가져와야하는 서버들을 모아두고 찾아서 추적하는 매커니즘
동적으로 변경되는 수천 대 서버 목록을 etcd, ZooKeeper로 실시간 추적.

• 안정 해시 링(Consistent Hash Ring)

다중 수집기 클러스터 간 타깃 서버의 중복 수집을 방지하고 작업 분담.



푸시 모델 (Push Model)

대표: CloudWatch, Graphite

동작 원리 (설명)

모니터링 대상 서버에 수집 에이전트를 설치하여, 생산되는 지표 데이터를 중앙 수집기 클러스터로 직접 능동 전송(Push)하는 방식입니다.

핵심 특징 및 요구 기술

• 생존 기간이 짧은 배치(Batch) 프로세스 최적

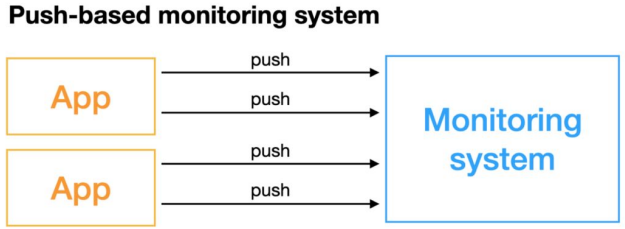
실행이 수 초 내에 종료되는 일회성 배치 작업은 풀링전에 종료될 수 있음,
작업이 끝나는 시점에 지표를 중앙 서버로 보내고 종료할 수 있어 위험 부담이 낮음.

• 유연한 멀티 데이터센터 네트워크

수집기 앞 로드밸런서만 열려 있으면 방화벽에 구애받지 않고 어디서나 수집.

• 에이전트 1차 집계 및 Auto-scaling

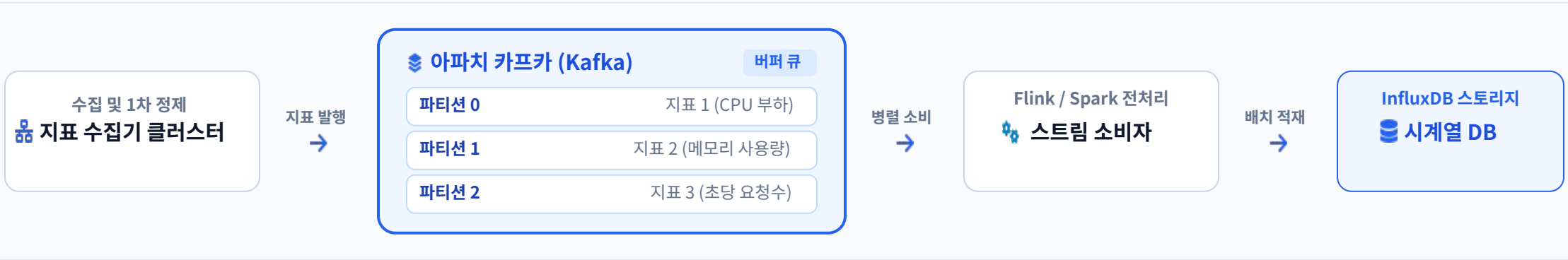
지표를 즉시 보내지 않고, 메모리에 모았다가 1차적으로 요약/집계하여 일정 주기마다 묶어서 전송이 가능.
수집기 CPU 부하 기반 자동 확장이 비교적 수월.



책 3단계 · 그림 5.15 & 5.16 기반 파이프라인

상세 설계 2: 카프카(Kafka)를 통한 파이프라인 확장 및 안정성 확보

데이터베이스에 장애가 발생한다면?



결합도 분리 및 무유실 보장

• 수집 컴포넌트와 처리 컴포넌트의 결합도를 완전히 분리합니다.
• 시계열 DB 장애 시에도 메시지는 카프카에 안전하게 보관되어 데이터가 유실되지 않으며, 복구 후 Flink/Spark 소비자가 재개합니다.

효율적인 파티셔닝 전략

• 지표 이름이나 태그/레이블을 파티션 키로 지정하여 대역폭에 맞춰 분배합니다.
• 다운스트림 스트림 소비자가 특정 지표 단위로 묶어서 병렬 집계 연산을 손쉽게 수행할 수 있습니다.

카프카 대안: Gorilla TSDB

• 상용 규모의 카프카 클러스터 구축/운영 비용이 부담스러운 경우의 대안입니다.
• 페이스북 Gorilla처럼 네트워크 장애 시에도 높은 쓰기 가용성을 자체 보장하는 인메모리 TSDB를 활용하면 큐 없이도 안정성을 유지합니다.
-> 최신 데이터 전용 인메모리 쓰기 스루(Write-through) 캐시 DB

저장 용량 최적화

상세 설계 3: 저장 용량 3단계 최적화 전략

1 이중-델타 인코딩

t_0: 1610087371 t_1: 1610087381 t_2: 1610087391

$\Delta_1 = +10$초 → $\Delta_2 = +10$초

차이의 변화(D) = 0 (단 1~4비트 저장)

32비트 → 4비트 수준 압축

절대 타임스탬프 원본 대신 기준 시각과의 차이(Delta) 및 차이의 변화율만 가변 길이 비트로 압축하여 용량을 대폭 절감합니다.

t_0: 11000000... Δ1: 0000001010 t_2: 0

2 단계별 다운샘플링

~ 7일 이내	원본 해상도 (초 단위)
7일 ~ 30일	1분 단위 집계 (Rollup)
30일 ~ 1년	1시간 단위 변환

시간 경과별 데이터 밀도 축소

시간이 지난 데이터는 미세한 초 단위 수치가 필요 없으므로, 평균/최댓값 단위로 롤업하여 스토리지 요구량을 수십 분의 1로 줄입니다.

3 저비용 냉동 저장소

Hot Tier (최근 30일) SSD / TSDB

↓ 비활성 데이터 배치 이관

Cold Tier (30일 ~ 1년) AWS S3 / 객체 스토리지

인프라 운영 비용 90% 절감

조회 빈도가 극히 낮은 장기 비활성 데이터를 값비싼 TSDB 디스크에서 분리하여 저렴한 객체 스토리지(Cold Storage)에 격리 보관합니다.

책 3단계 · 그림 5.19 & 5.20 기반 상세 아키텍처

상세 설계 4: 경보 관리자(Alert Manager) 파이프라인 및 시각화

1. 규칙 설정 및 질의

- YAML 파일로 경보 규칙 정의 후 캐시에 로드
- 질의 서비스를 주기적으로 호출하여 임계치 검사

핵심 컴포넌트

2. 경보 관리자 (Alert Manager)

- **병합 및 중복 제거:** 동일 인스턴스 다중 이벤트를 1개로 합쳐 알림 폭풍 방지
- **접근 제어 & 재시도:** 휴먼 에러 방지를 위한 역할 기반 권한 제어하고 알림 전송 실패 시 재시도

3. 경보 저장소 (Alert Store)

- 카산드라 형태의 분산 키-값 저장소
- 상태(대기, 발령, 해결) 영속화로 **최소 1회 전달(At-least-once)** 보장

4. 카프카 및 다중 발송 채널

- 이메일
- SMS
- PagerDuty
- Webhook

시각화 및 대시보드: 빌드 vs 구매 (Build vs Buy)

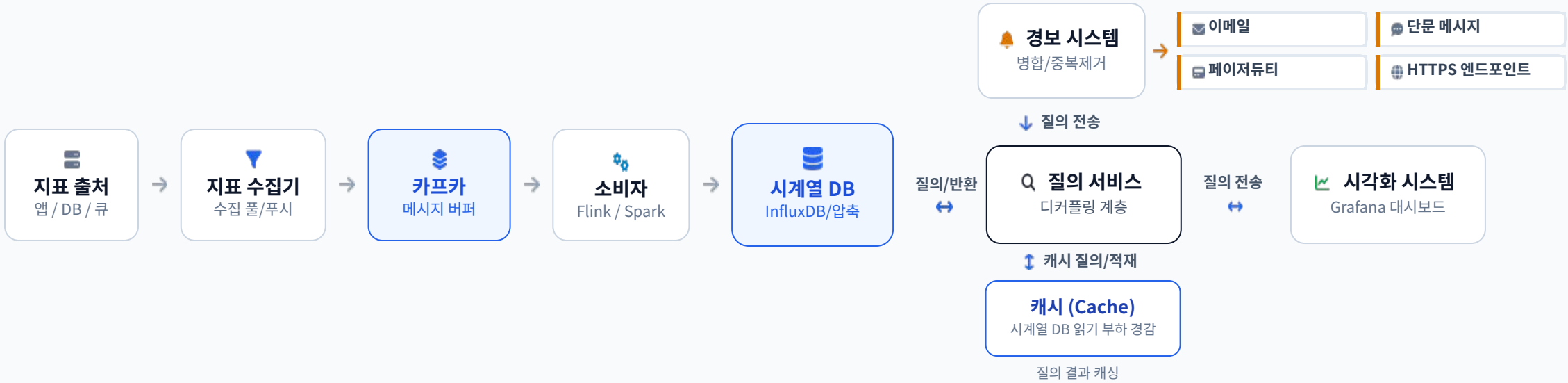
실무에서는 복잡한 대시보드를 자체 개발하기보다, TSDB 플러그인이 풍부한 **그라파나(Grafana)**를 채택하는 것이 안정성과 비용 측면에서 표준입니다.

Grafana 실무 채택

4단계: 마무리 · 그림 5.22 최종 설계안

최종 아키텍처

① 풀/푸시 수집 선정 | ② 카프카 버퍼 확장 | ③ 시계열 DB(TSDB) | ④ 3단계 다운샘플링 | ⑤ 경보 병합 & Grafana



최종 아키텍처 완성: 카프카 버퍼링으로 DB 장애 시에도 지표 유실을 원천 차단하고, 전용 질의 서비스와 캐시 계층을 두어 대시보드와 경보 평가의 저지연 성능을 완벽히 보장합니다.

확장 설계 로드맵 · 관측 가능성(Observability) 3대 축의 완성

다음 설계 과제: 로그 및 분산 추적 시스템의 확장 요구사항

■ 인프라 기본 환경 (기준 일치) — DAU 1억 명 · 1,000개 서버 풀 × 100대 서버 (총 10만 대 노드 규모)

■ 대규모 중앙 로그 시스템 (Logs)

비정형 텍스트

데이터 규모 및 인입량

초당 1,000만 줄 (10M logs/sec) · 일일 약 430TB 인입

핵심 과제

- **PII 마스킹:** 개인식별정보(주민/카드번호) 실시간 비식별화
- **텍스트 구조화:** 다양한 포맷의 JSON 파싱 및 필드 정규화
- **로그 폭풍 제어:** 장애 시 에이전트 Rate Limiting & 배압 제어

스토리지 & 보관 정책

역색인 비용 통제(청크 압축) · 핫(7일)-웜(30일)-콜드(1년 S3) 계층화

■ 대규모 분산 추적 시스템 (Tracing)

마이크로서비스 호출 경로

트래픽 규모 및 스팬(Span) 인입

10만 RPS (호출 깊이 15) · 초당 170만~450만 Span

핵심 과제

- **Trace-ID 전파:** HTTP/gRPC 헤더 및 Kafka 비동기 컨텍스트 전파
- **워터폴 재조립:** 흩어진 스팬 조각들의 비동기 DAG 트리 복원
- **서비스 무간섭:** 애플리케이션 지연 영향 1% 미만(1ms 이하) 유지

샘플링 전략 & 보관 주기

정상 요청 1% 수집 vs 에러(5xx) · P99 지연 100% 테일 샘플링 (7~14일 보관)

관측성 통합 로드맵: 지표(Metrics)의 실시간 임계치 감시를 기반으로, 분산 추적(Tracing)으로 병목 구간을 특정하고, 로그(Logs)로 세부 에러 원인을 규명하는 완성형 관측 파이프라인으로 확장합니다.

설계 마무리

대규모 지표 모니터링 시스템 완성

수집 계층
서비스 탐색 + 풀 모델

전송 버퍼
카프카 파티셔닝

저장 최적화
TSDB + 다운샘플링

경보 및 뷰
중복 제거 + Grafana

감사합니다. Q&A